Introduction to HPC

Dimitrios Papadopoulos
Associate Professor, DTU Compute

Deep Learning





Training and deploying a deep learning model

GPU is a must!!!

Options:

- Your CPU in your local machine (technically yes, but you will finish the assignments after Christmas ☺)
- Google Colab (a free GPU, but not really…)
- The expensive GPU of your gaming desktop (technically yes, but you work in groups at campus)
- A GPU cluster (DTU has HPC cluster and we have allocated 16 10GB slices from high-end GPUs (cost about 1M DKK) for this course)

Let’s do a quick poll

(raise your hand if you agree)



(1) I am familiar with basic UNIX commands

*ls -l, pwd, cp, scp, rm -rf *,cd .., mkdir, sudo, cat, rsync, df, grep, vim, find, netstat*

(2) I know how to access a server via ssh

`ssh user@awesomeserver.dtu.dk`

(3) I have used colab or jupyter notebook

jupyter notebook --no-browser --port=9999

(4) I have used pip, conda and/or virtualenv

```
conda create --name notmyfirstconda
python=3.5
```

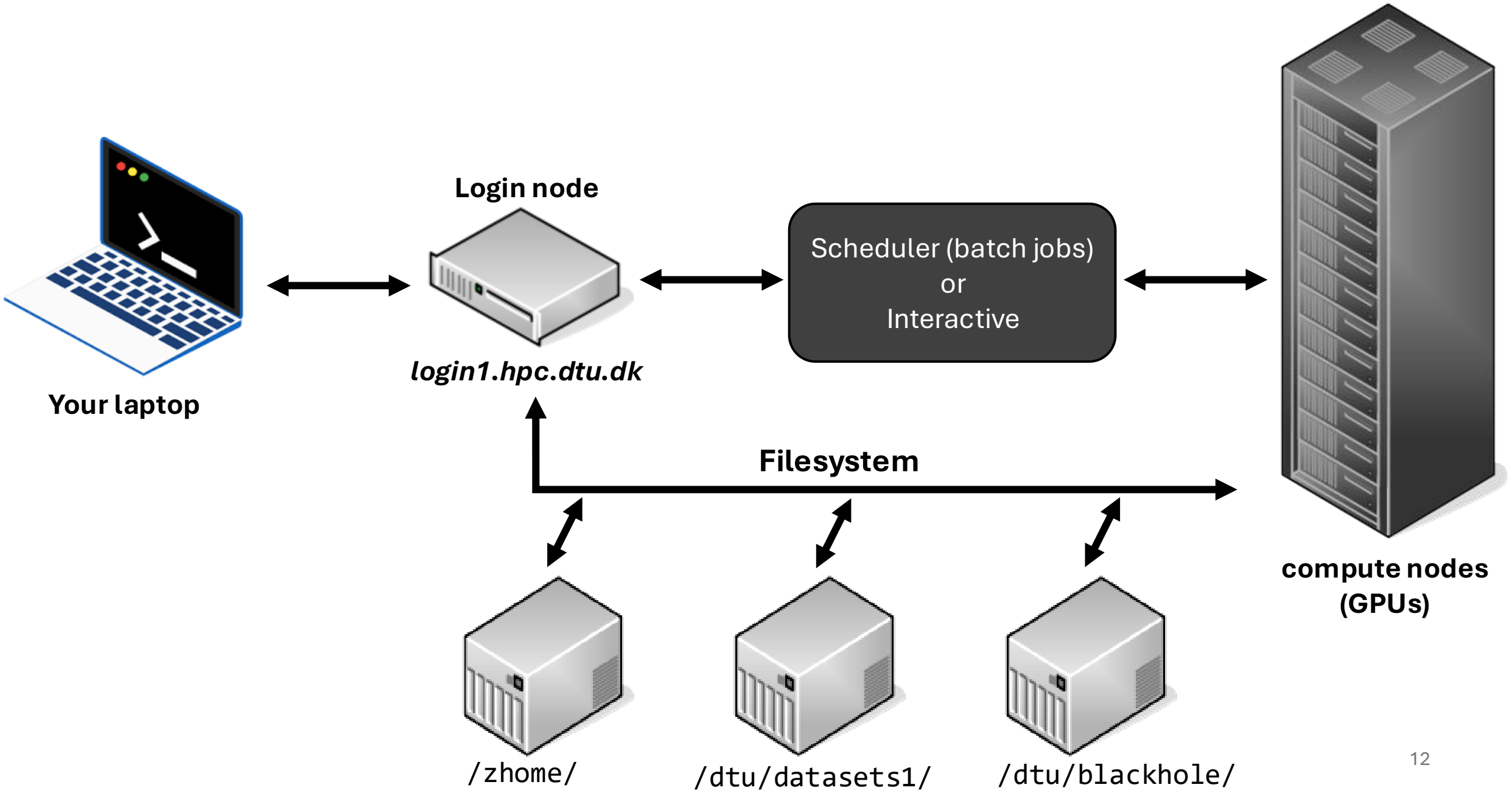
(5) I am familiar with running scripts on a cluster (batch or interactive) using a GPU

Steps

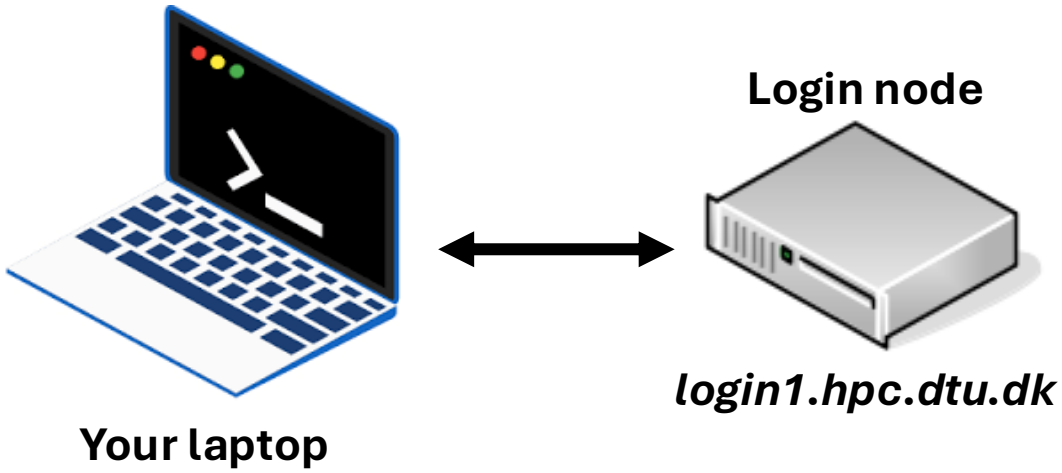
1) Terminal
2) Learn Basic UNIX commands
3) Access HPC
4) Interactive nodes and Batch jobs
6) Copy something to HPC filesystem
7) Monitoring commands and filesystem
8) Loading modules
9) Run a script (+GPU)
10) Conda or virtual environment
11) Batch jobs and submitting jobs
12) Jupyter notebooks
13) Extra tips and tools



HPC


Login node
Scheduler (batch jobs)
or
Interactive
Your laptop
login1.hpc.dtu.dk
Filesystem
compute nodes
(GPUs)
/zhome/
/dtu/datasets1/
/dtu/blackhole/

(1) Terminal



(1) Terminal



Unix-based: default terminal or iTerm

Windows: deault in new windows (?) or Putty, Kitty, windows terminal

(2) Basic UNIX commands

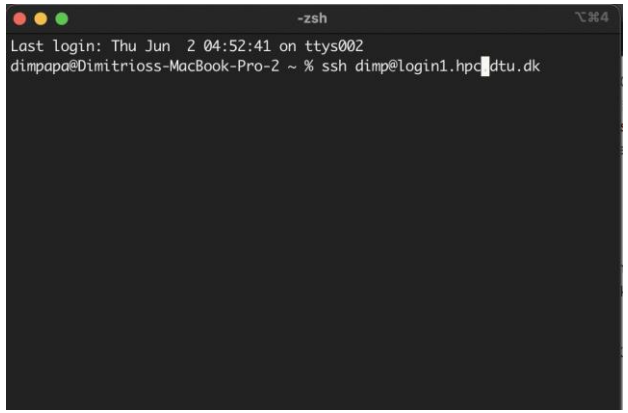
- Navigating to the filesystem
- Copy-Move-Create
- df/ du
- top/ htop
- man/ help
- nvidia-smi/ gpu-stat
- quota
- Googling for the rest if needed



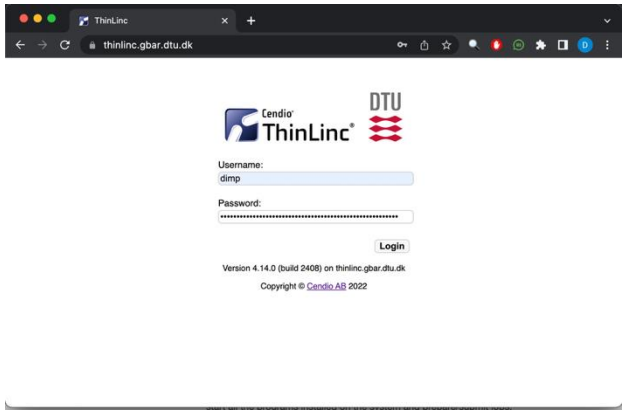
https://www.unixtutorial.org/basic-unix-commands

(3) Access HPC

- Everything about HPC: https://www.hpc.dtu.dk/
- Accessing the LSF 10 Cluster: https://www.hpc.dtu.dk/?page_id=2501



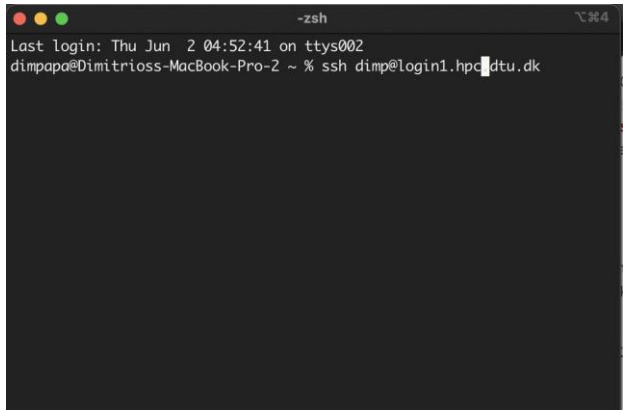
command line



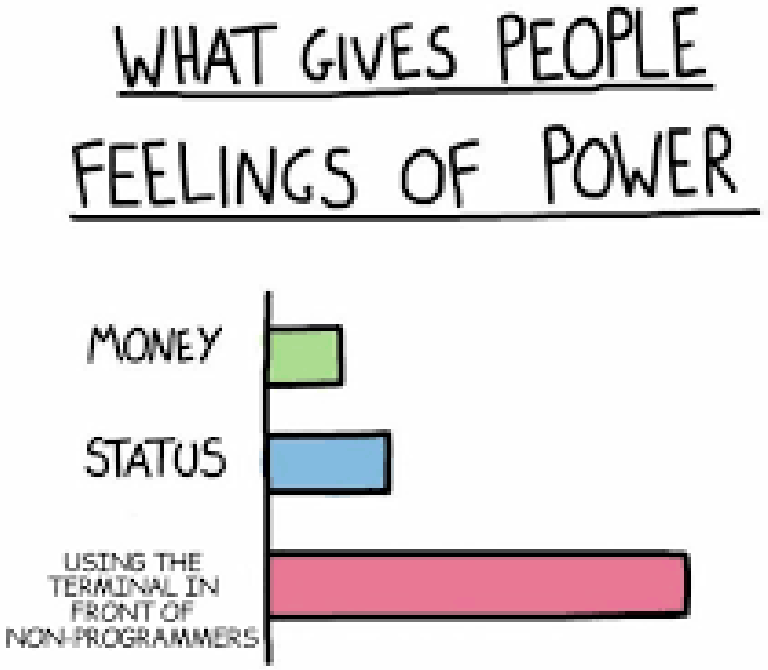
thinlinc graphics

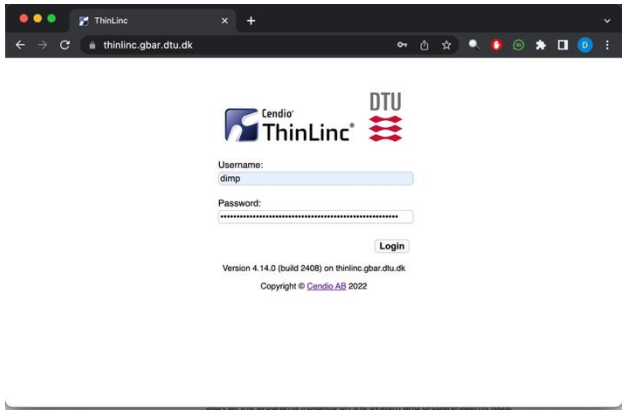
(3) Access HPC

- Everything about HPC: https://www.hpc.dtu.dk/
- Accessing the LSF 10 Cluster: https://www.hpc.dtu.dk/?page_id=2501



command line





thinlinc graphics

(3) Access HPC

ssh-keys and VPN: https://www.hpc.dtu.dk/?page_id=4317




```
When connecting from _outside_ of DTU (and not using DTU-VPN)
one is required to authenticate with a
ssh-key + ssh-key-keyphrase + DTU-password.

From inside of DTU (or connecting via DTU-VPN) one can also connect
with password or ssh-key.

https://www.hpc.dtu.dk/?page_id=4317

---
(For thinlinc-users: This is not an error-message, this is an
"information-message", please press "close" in the
right-bottom-corner to continue.)
---
dimp@login1.hpc.dtu.dk: Permission denied (publickey).
```

(3) Access HPC

ssh-keys and VPN: https://www.hpc.dtu.dk/?page_id=4317

SSH and ThinLinc login using ssh-keys

To increase the security of the DTU infrastructure, the whole DTU network setup is changing, and this affects also the way users can access G-bar and the DCC HPC clusters ***from outside the DTU network***.

Nothing changes, if you are accessing G-bar or the HPC cluster either on Campus, or via the DTU VPN. The instructions below are only relevant, if you need to access those resources from **outside** the DTU network, and the VPN is not a viable option, or it affects the performance critically (e.g. increased latency, incompatibility with a pre-existent pipeline, significant data transfer, etc).

If you only work on Campus or via the DTU VPN

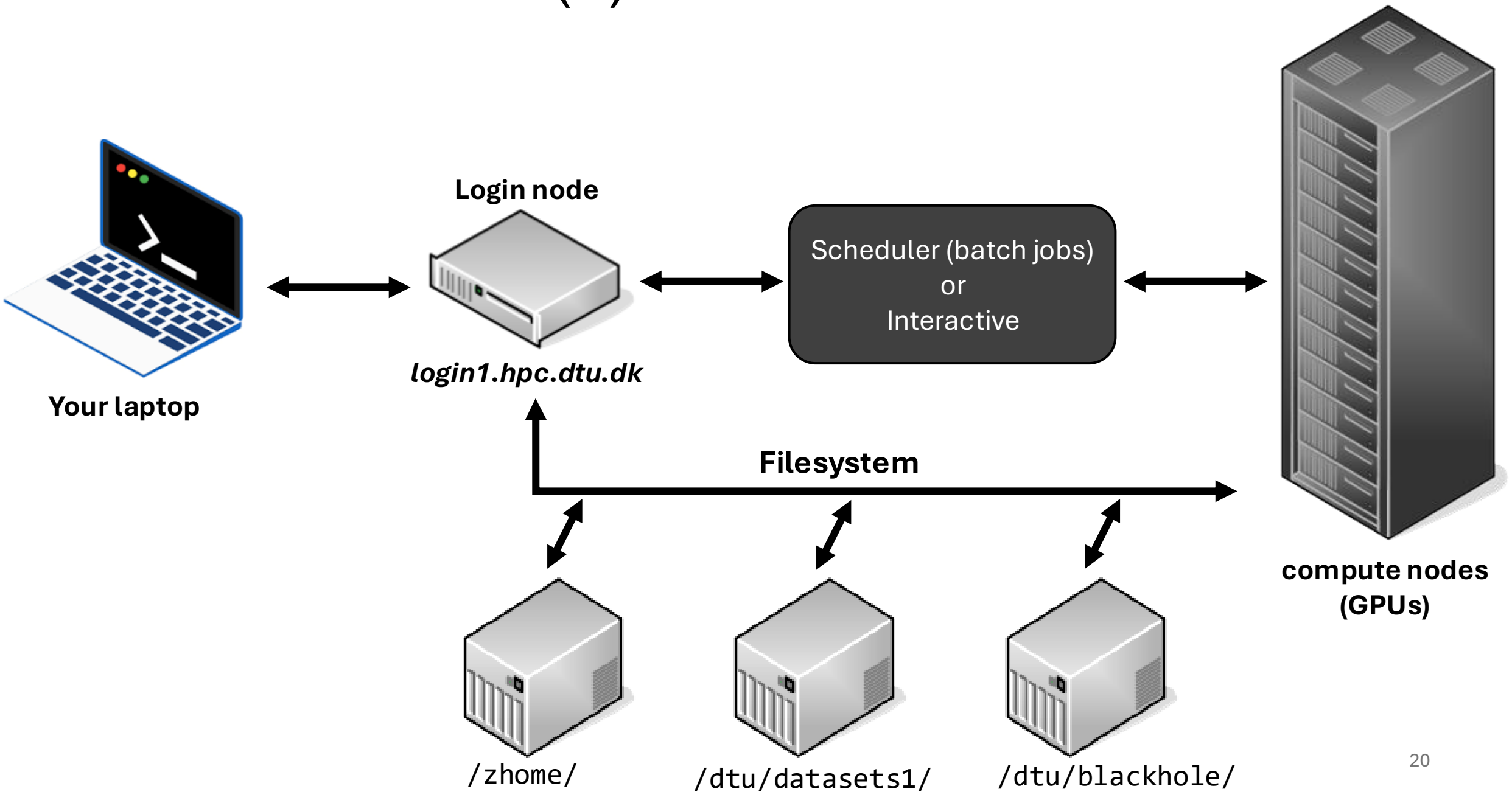
Nothing changes for you. Just make sure that your VPN setup is up to date, and the Multi Factor Authentication is working.
You can safely skip the rest of the page.

If you work remotely, and always using the VPN is not a convenient option

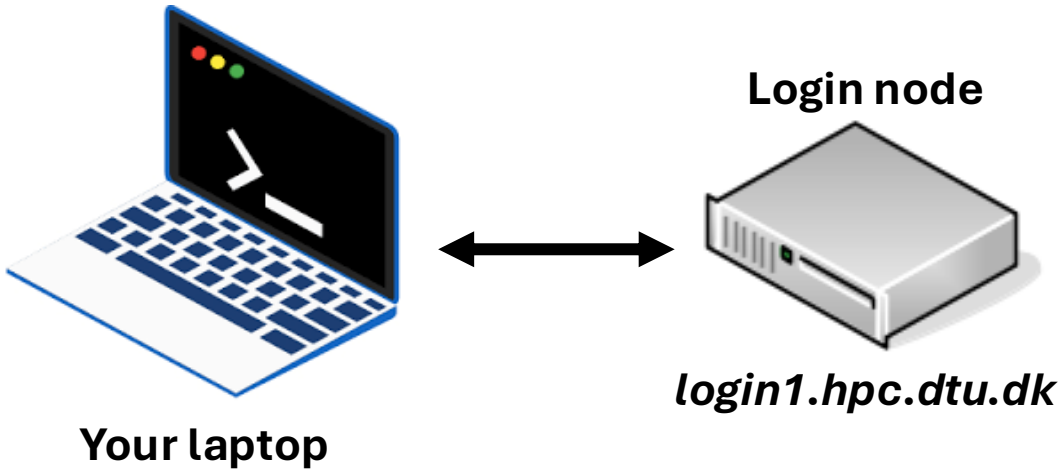
SSH access will from now on require the use of ssh-keys + ssh-key passphrase + DTU password.
Instructions on how to setup your machine(s), and how to configure SSH for your G-bar/HPC account follow below.

Note: the guide below requires that you – during the setup process – are either connected to a DTU network on Campus, or via VPN (see DTU Inside for the VPN setup). Remember that the CISCO VPN requires the Multi Factor Authentication.

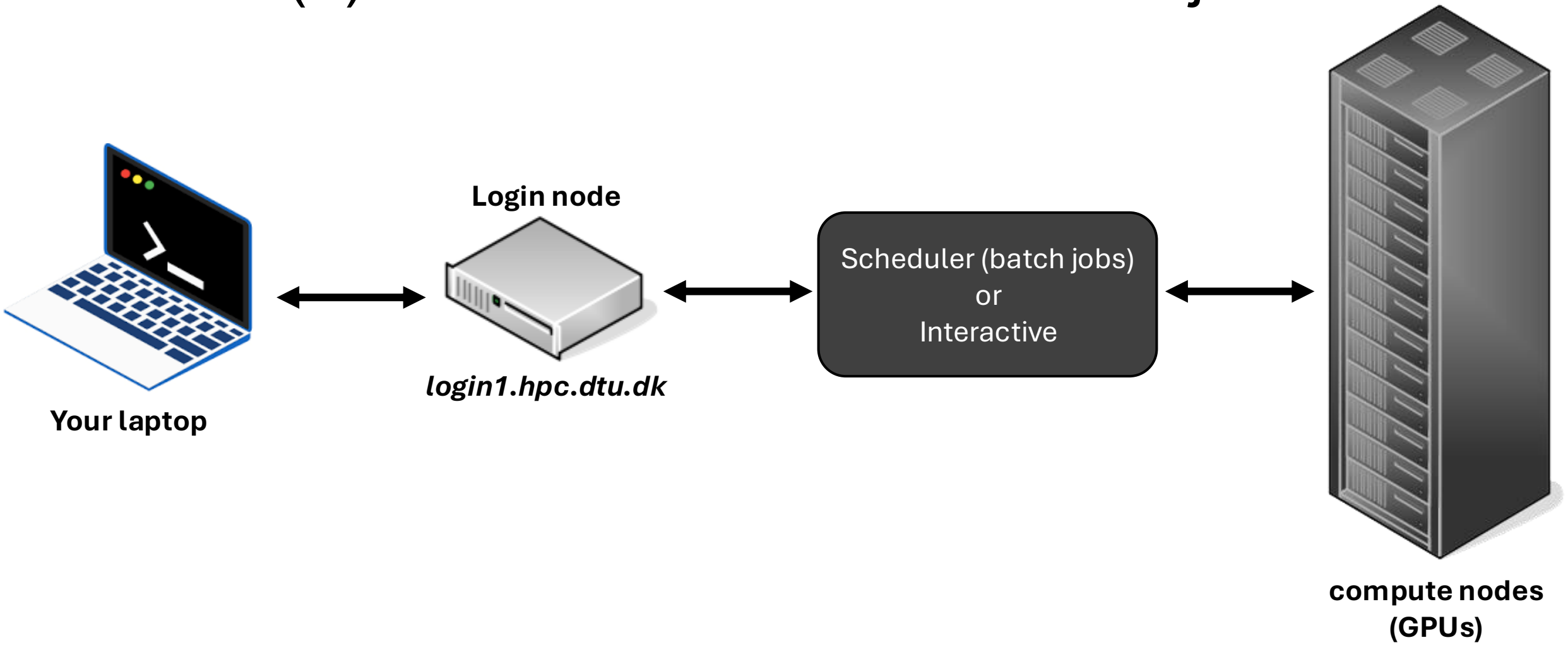
(3) Access HPC


Login node
Scheduler (batch jobs)
or
Interactive
login1.hpc.dtu.dk
Your laptop
Filesystem
compute nodes
(GPUs)
/zhome/
/dtu/datasets1/
/dtu/blackhole/

(4) Interactive nodes and Batch jobs



(4) Interactive nodes and Batch jobs



(4) Interactive nodes

02516sh

Only 1 at a time →

if you can’t open one→

kill the open session →

if not sure, *bkill XXXXX*

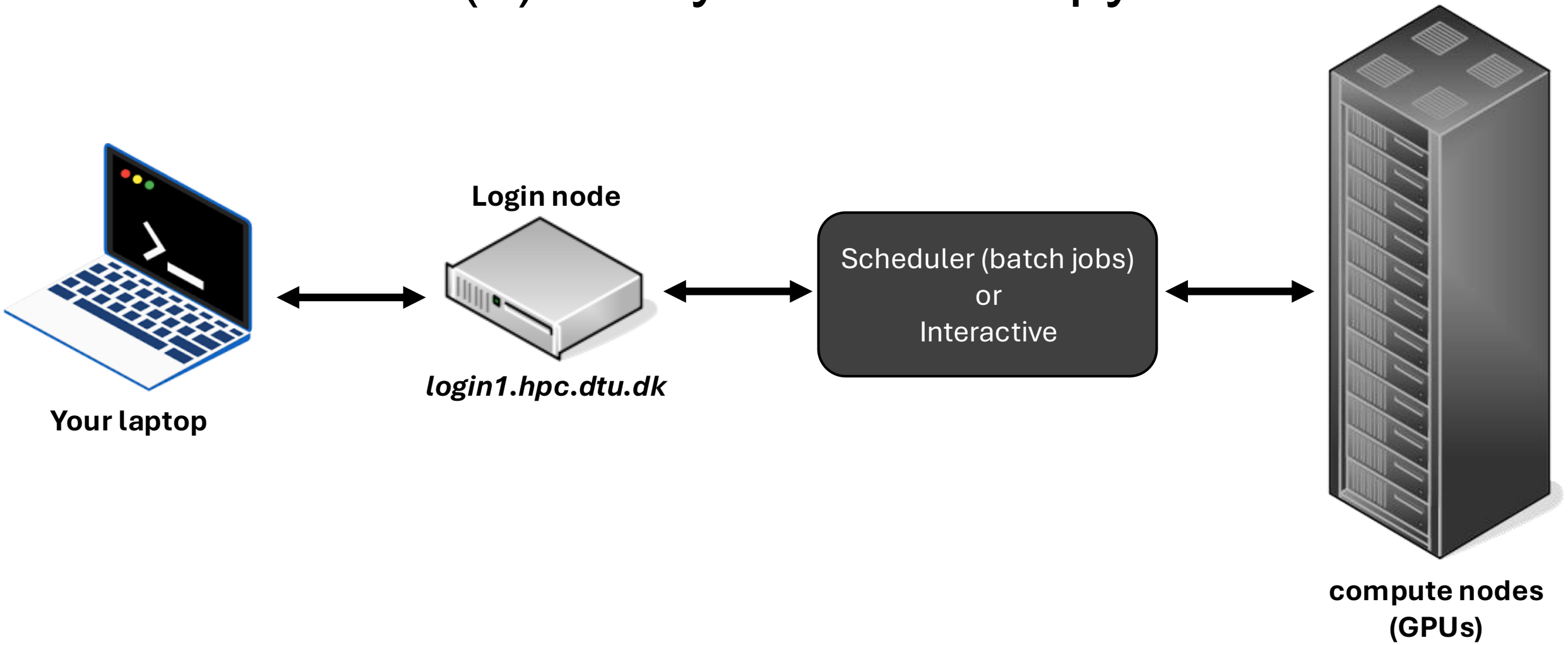
(4) Interactive nodes

02516sh

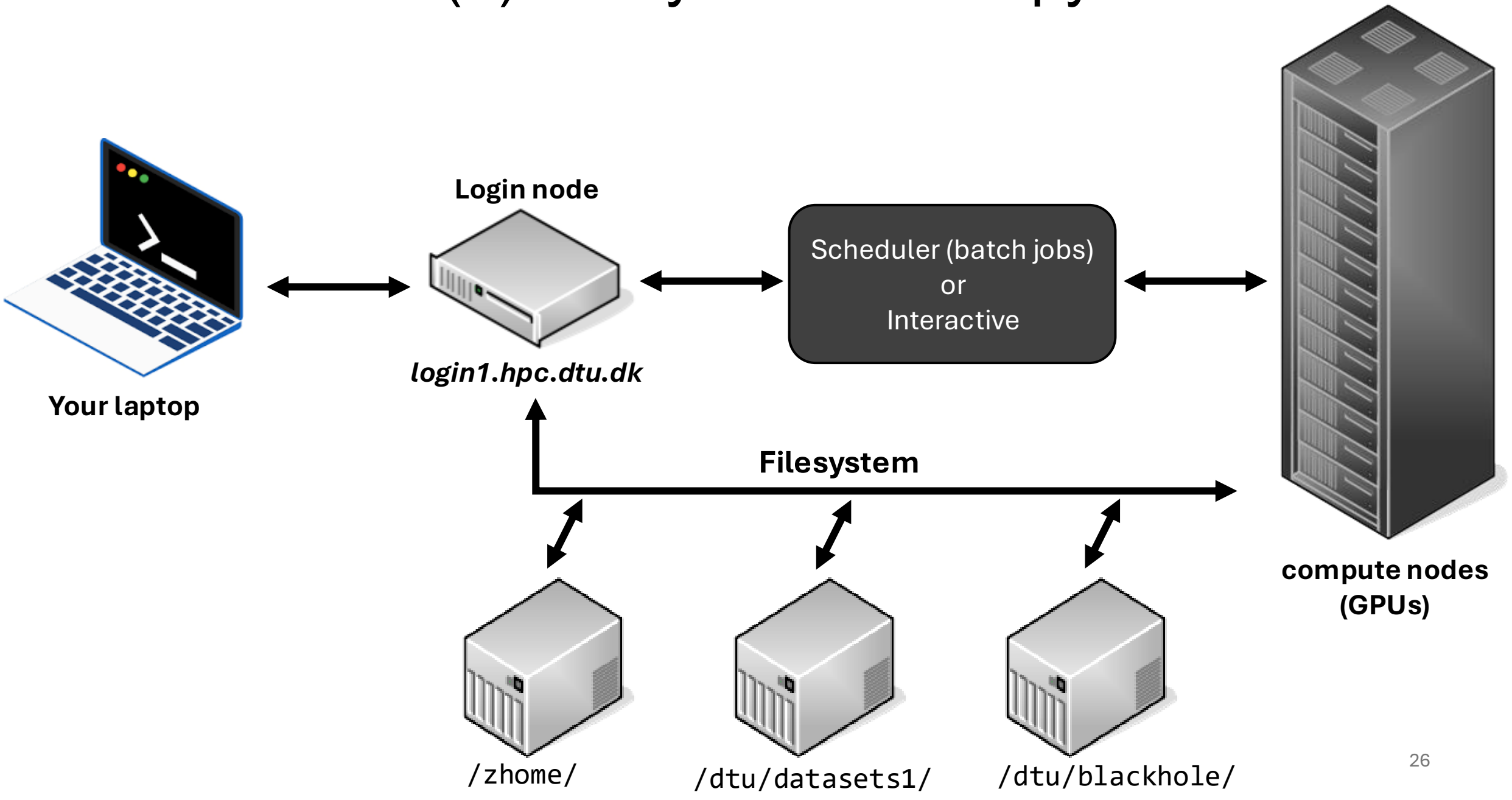
Only 1 at a time →
if you can't open one→
kill the open session →
if not sure, *bkill XXXXX*

- Mainly for **testing/debugging** purposes
- **Not a good practice** to run long scripts here
- If you close your terminal, you kill your run

(5) Filesystem and copy



(5) Filesystem and copy



(5) Filesystem and copy

- **Monitoring:**

  `getquota_zhome.sh`

- **Local machine to cluster:**

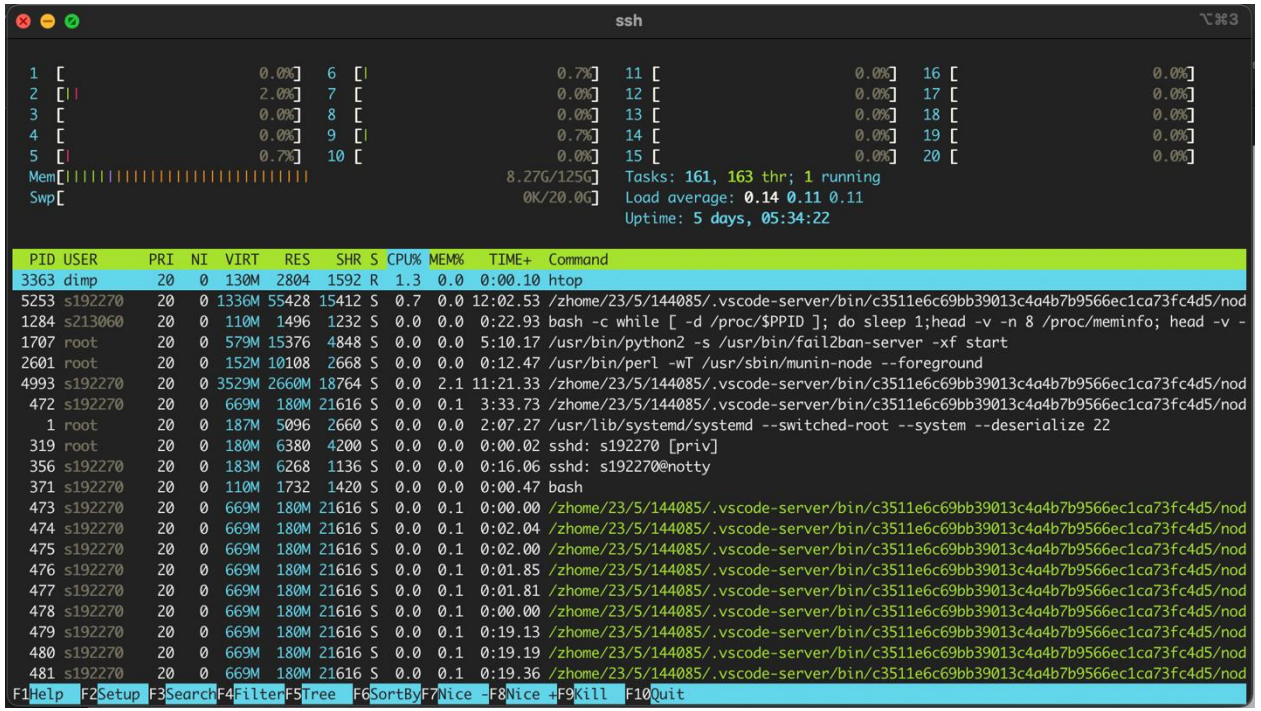
  ```
  scp randomimage.png
  dimp@login1.hpc.dtu.dk:/path/in/the/cluster/myfolder/bla/bla/
  ```

- **Shared space in cluster (read-only):**

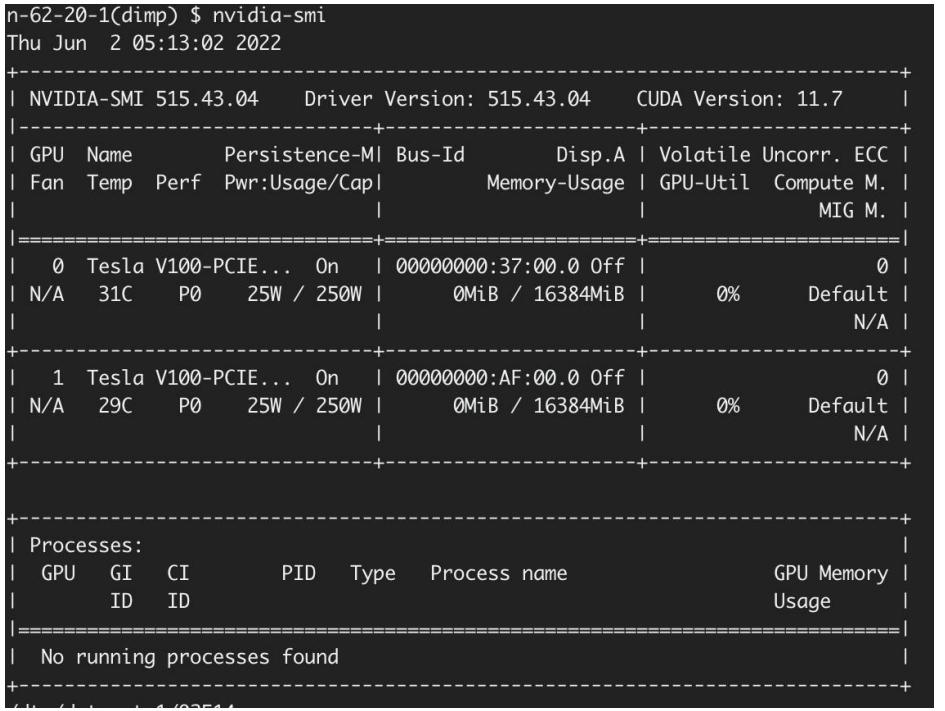
  `/dtu/datasets1/02514/`

(6) Monitoring

htop



nvidia-smi



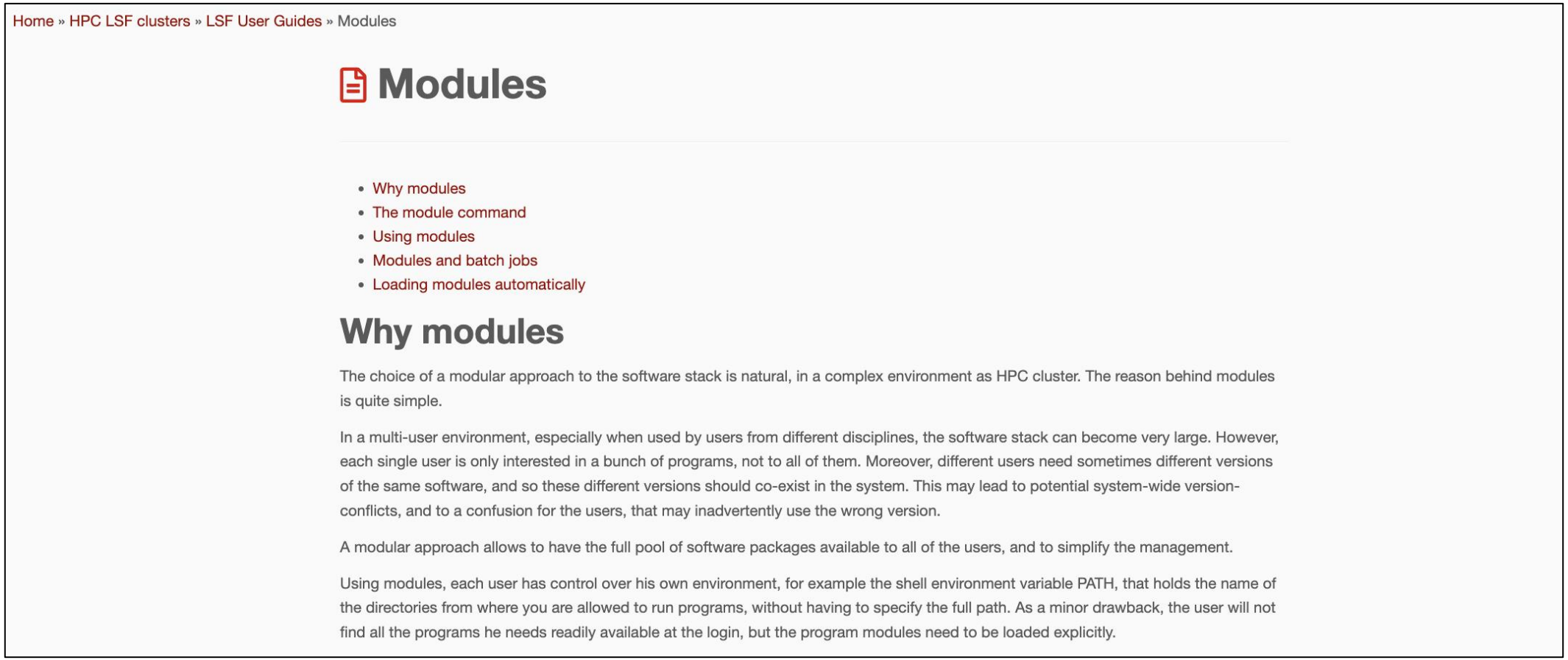
getquota_zhome.sh

```
n-62-20-1(dimp) $ getquota_zhome.sh
You are using 817.51M of 30.00G quota

(Information updated every 30 minutes. Last Update: 2022-06-02 04:49:15)
```

(7) Modules

Modules: https://www.hpc.dtu.dk/?page_id=282



Home » HPC LSF clusters » LSF User Guides » Modules

Modules

- Why modules
- The module command
- Using modules
- Modules and batch jobs
- Loading modules automatically

Why modules

The choice of a modular approach to the software stack is natural, in a complex environment as HPC cluster. The reason behind modules is quite simple.

In a multi-user environment, especially when used by users from different disciplines, the software stack can become very large. However, each single user is only interested in a bunch of programs, not to all of them. Moreover, different users need sometimes different versions of the same software, and so these different versions should co-exist in the system. This may lead to potential system-wide version-conflicts, and to a confusion for the users, that may inadvertently use the wrong version.

A modular approach allows to have the full pool of software packages available to all of the users, and to simplify the management.

Using modules, each user has control over his own environment, for example the shell environment variable PATH, that holds the name of the directories from where you are allowed to run programs, without having to specify the full path. As a minor drawback, the user will not find all the programs he needs readily available at the login, but the program modules need to be loaded explicitly.

(8) Run a script

```
python i_can_run_hello_world_remotely.py
```

```
CUDA_VISIBLE_DEVICES=1 python i_can_even_run_it_on_a_gpu_that_i_select_wow.py
```

Live demo Part 1!



(9) Conda or Virtualenv

Virtual environments

Using virtual environments lets you have different stacks of software packages in different folders. Please search the web for additional information regarding virtual environments.

First, select the Python version you want, and load the corresponding module. Here we choose python 3.8.2. If you use versions older than 3.6, the instructions to create the virtual environment may differ.

```
module load python3/3.8.2
```

Then create a virtual environment. We choose the name `venv_1`

```
python3 -m venv venv_1
```

That's all. Now, activate the environment with the command

```
source venv_1/bin/activate
```

When the environment is active the shell notes this to the left of the cursor. It will look something like this:

```
(venv_1) ~
gbarlogin1$
```

This indicates the virtual environment name in parenthesis.

Note that *both* the commands `python` *and* `python3` point to the python version of your original module. Now, when the environment is active, you can install all the packages you need with the normal `python3 -m pip install ...` syntax, for example (note `--user` should **NOT** be added)

```
python -m pip install scikit-learn
```

All those packages will be installed locally under the directory

```
venv_1/lib/python3.8/site-packages/
```

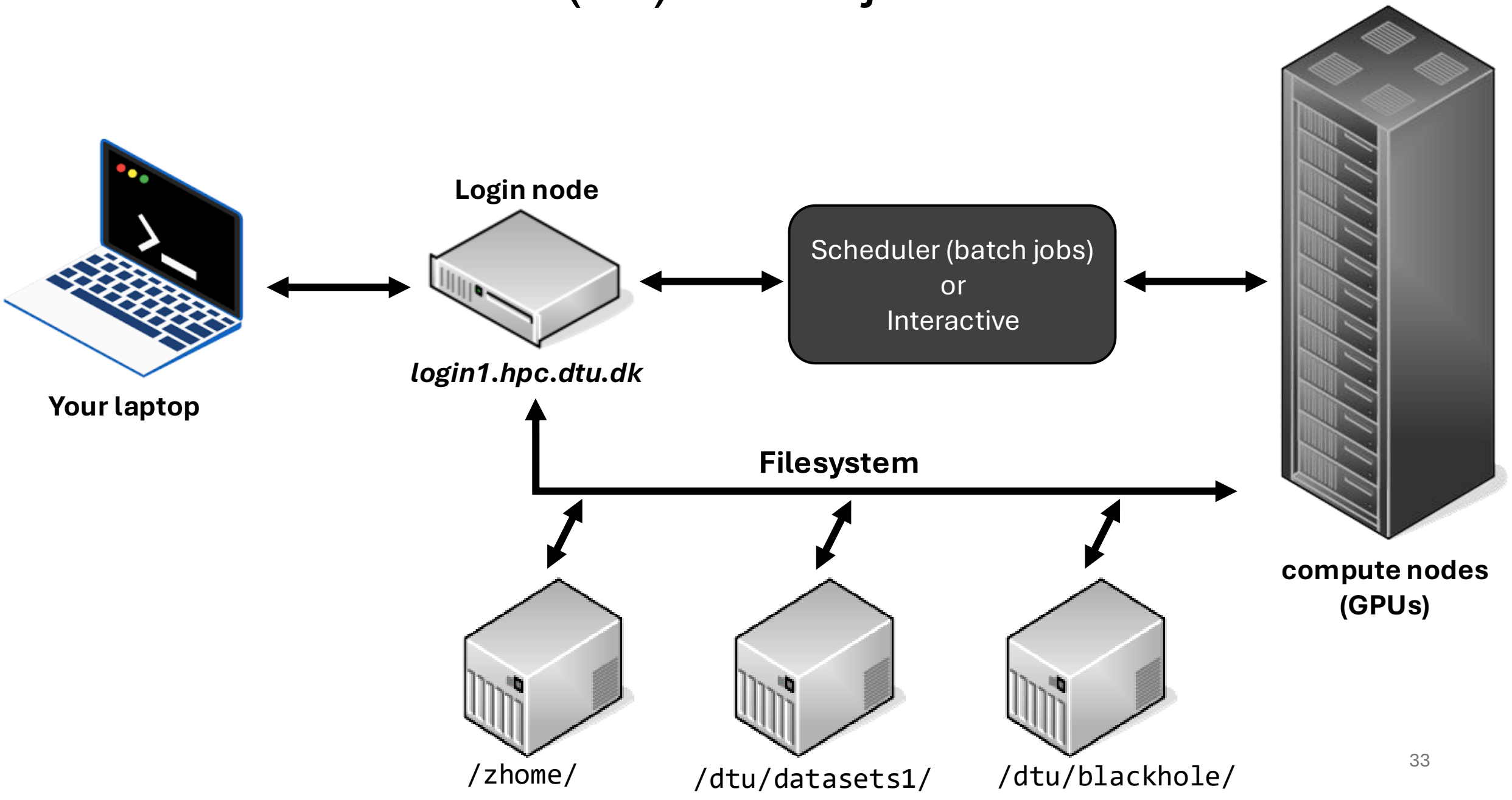
When you have finished to use the environment, just deactivate it with the command

```
deactivate
```

https://www.hpc.dtu.dk/?page_id=3678



(10) Batch jobs



(10) Batch jobs

- Batch jobs: https://www.hpc.dtu.dk/?page_id=1416
- Managing jobs: https://www.hpc.dtu.dk/?page_id=1519
- GPUs under LSF: https://www.hpc.dtu.dk/?page_id=2759

IBM Spectrum LSF: *workload management platform, job scheduler, for distributed high performance computing by IBM*

(10) Batch jobs - steps

- My **python script** is ready: main.py
- I have a **virtualenv** with all needed packages properly installed
- I create a **bash script** to submit my job
- I **submit** the job and I can monitor the progress

(10) Batch jobs – bash script

```
#!/bin/sh
### ------------- specify queue name ----------------
#BSUB -q c02516

### ------------- specify gpu request----------------
#BSUB -gpu “num=1:mode=exclusive_process”

### ------------- specify job name ----------------
#BSUB -J testjob

### ------------- specify number of cores ----------------
#BSUB -n 4
#BSUB -R "span[hosts=1]"

### ------------- specify CPU memory requirements ----------------
#BSUB -R "rusage[mem=20GB]”

### ------------- specify wall-clock time (max allowed is 12:00)---------------- #BSUB -W 12:00

#BSUB -o OUTPUT_FILE%J.out
#BSUB -e OUTPUT_FILE%J.err

source /PATH/TO/ENVIRONMENT
python /PATH/TO/PYTHON_FILE
```

(10) Batch jobs – submit job

bsub -app c02516_1g.10gb < jobscript.sh

bstat

```
JOBID      USER    QUEUE      JOB_NAME   NALLOC STAT  START_TIME      ELAPSED
22843021   dimp    gpu02516i  02516sh         1 RUN   Oct  2 10:01    1:27:25
```

(10) Batch jobs – submit job

bsub -app c02516_1g.10gb < jobscript.sh

bstat

```
JOBID      USER    QUEUE      JOB_NAME   NALLOC STAT  START_TIME      ELAPSED
22843021   dimp    gpu02516i  02516sh         1 RUN   Oct  2 10:01    1:27:25
```

More GPU per job??? (not recommended unless is needed)

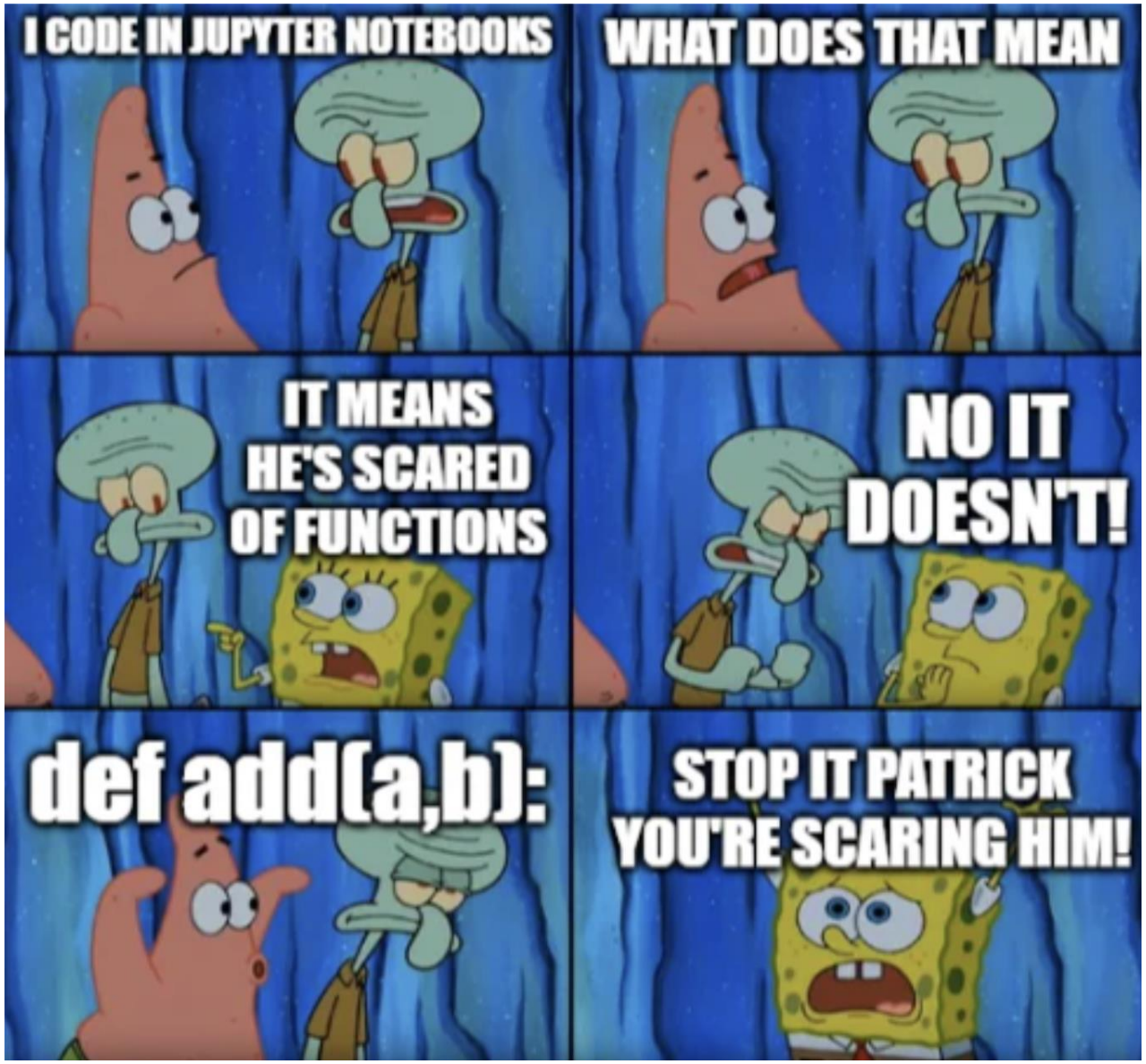
bsub -app c02516_2g.20gb < jobscript.sh

bsub -app c02516_4g.40gb < jobscript.sh

Live demo Part 2!



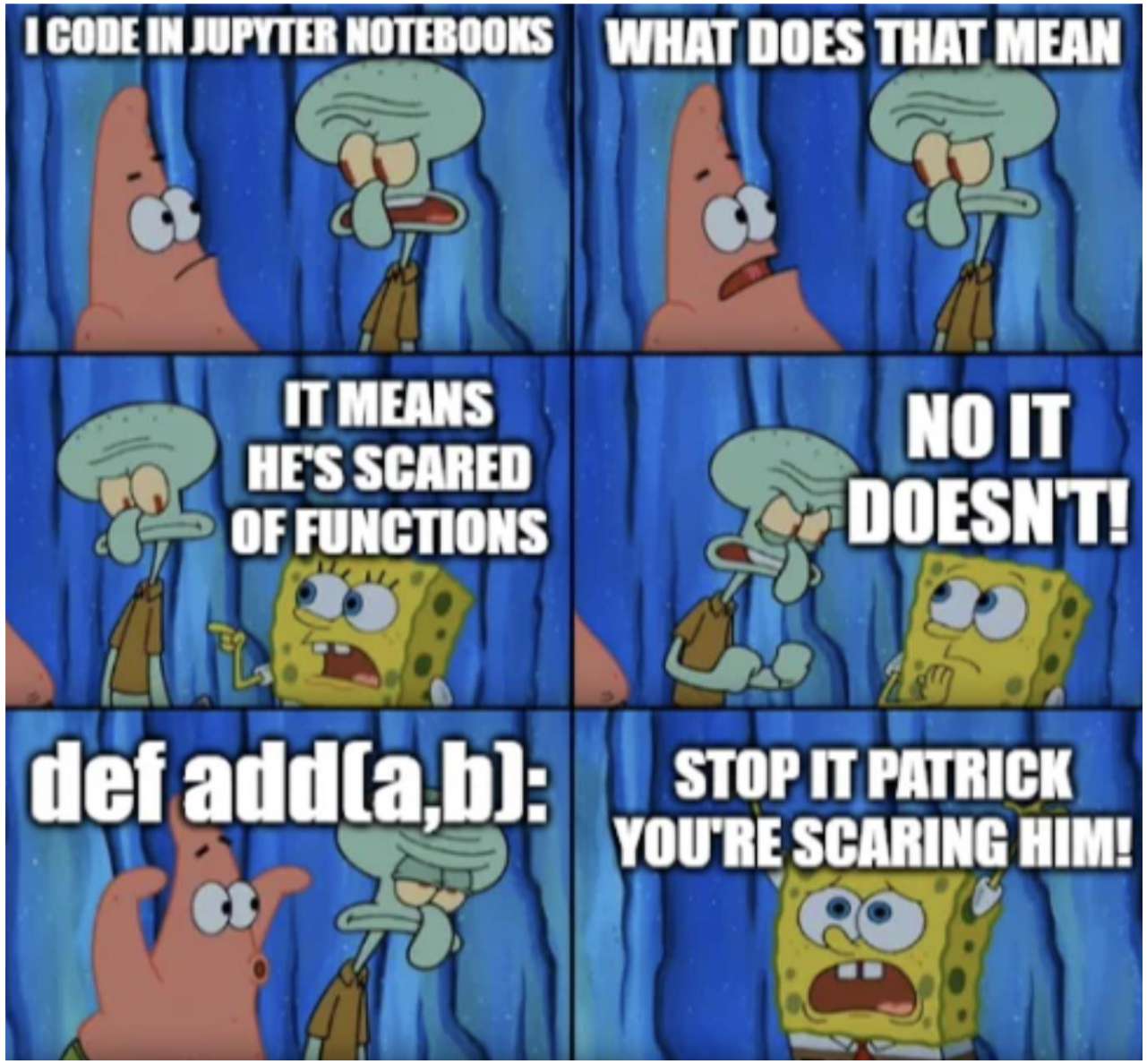
(11) Jupyter vs scripts



+ Interactivity

+ Data visualization

(11) Jupyter vs scripts



+ Interactivity

+ Data visualization

- Automation

- Reproducibility

- Version control

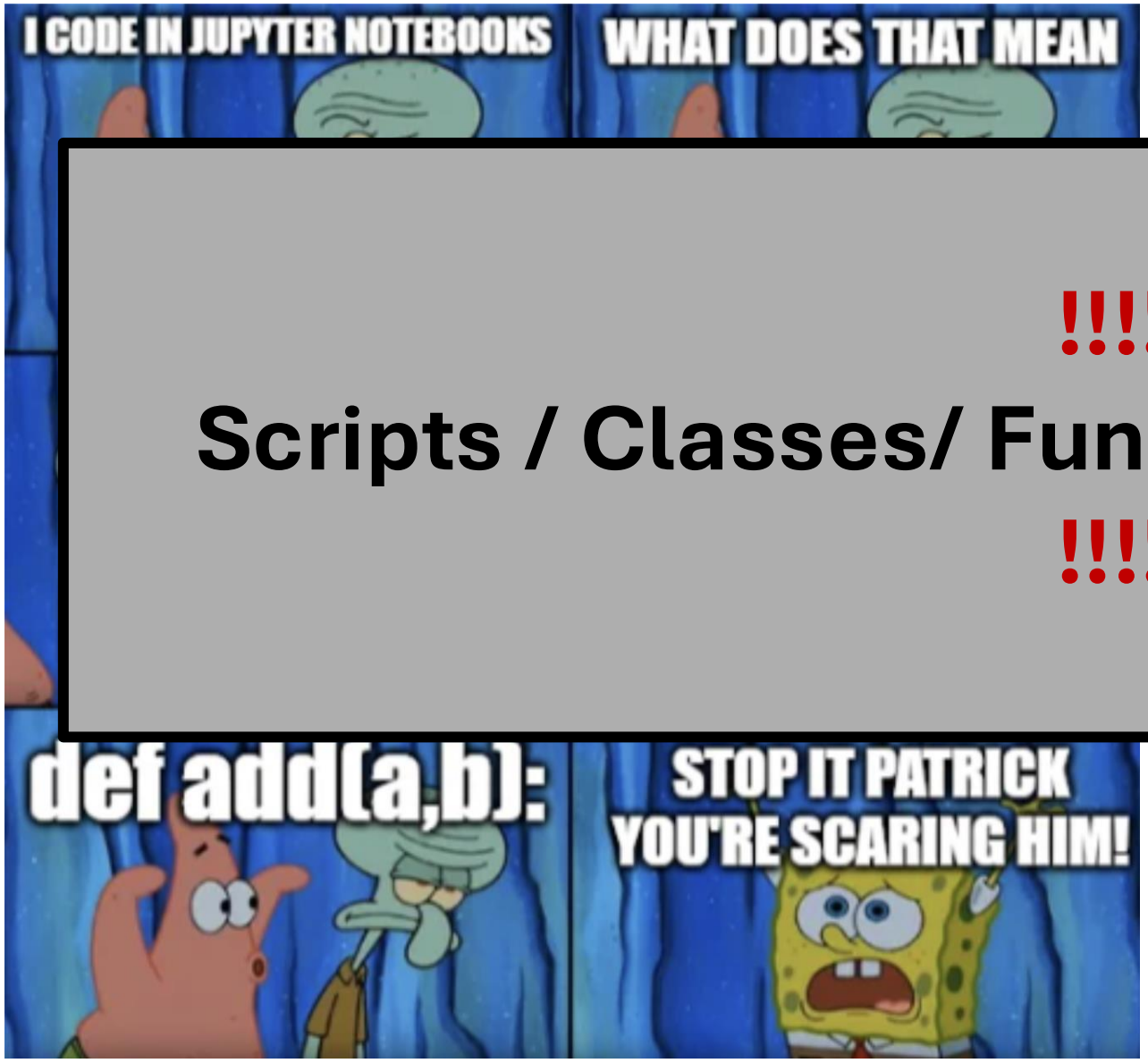
- Scalability

- Memory efficiency

- No test support

- No git support

(11) Jupyter vs scripts



+ Interactivity

!!!!!!

Scripts / Classes/ Functions >>> Notebooks

!!!!!!

- Memory efficiency

- No test support

- No git support

Total resources



Interactive node
(02516sh)

2 A100 GPUs (40GB each)

Batch node
(c02516)

16 GPU-slices (10GB each)

About 10GB GPU per group

Extra tips

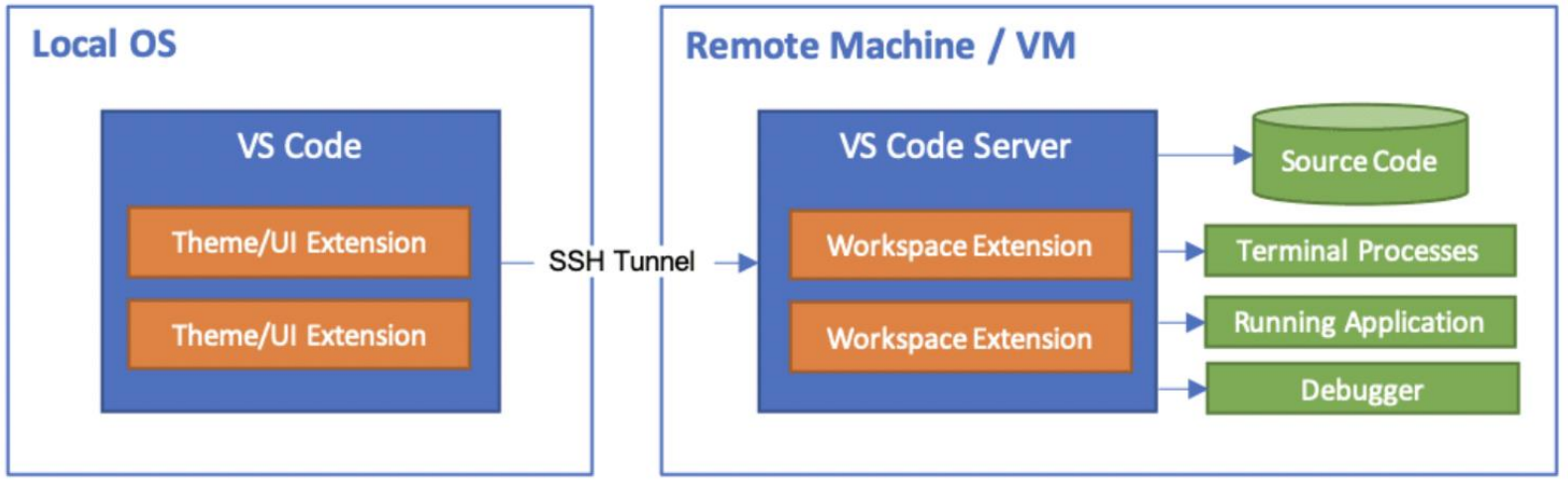
Scripts / Classes/ Functions >>> Notebooks

BUT

Editors>>>vim

Vscode and Vscode with remote ssh:

https://code.visualstudio.com/docs/remote/ss



Extra tips

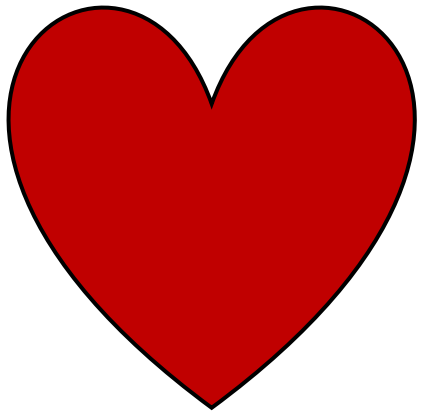
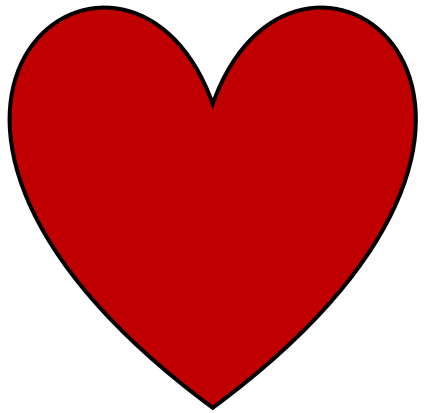
No group shared folder!





Extra tips

“My terminal is awesome!”

 **tmux** 

Slack channel >> HPC support

join #hpc-compute

Questions???

- **Exercise 2.1: HPC**

  Access HPC and run a simple script

- **Project 2: Video Classification**

  Do not run notebooks! Submit batch jobs instead!

Good Luck!